COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 DATA STRUCTURES AND ALGORITHMS	**SECTION:**	CPE21S3
DATE PERFORMED:	SEPTEMBER 8 2026	**DATE SUBMITTED:**	SEPTEMBER 10 2026
NAME:	DUSTIN CARL B. MAPAS	**INSTRUCTOR:**	ENGR. JIMLORD QUEJADO

ACTIVITY NO. 7.1	**Hands-on Activity 7.1 Sorting Algorithms Pt1**

1. PROCEDURE OUTPUT

Code + Console Screenshot	Code:
	`#include <iostream>`
	`#include <cstdlib>`
	`#include <time.h>`
	`#include "sortingAlgo.h"`
	
	`const int maxSize = 100;`
	
	`void displayAll(int arr[]);`
	
	`int main() {`
	
	`int unsortedArr[maxSize];`
	
	`srand( Seed: time(NULL));`
	
	`for (int i = 0; i < maxSize; i++) {`
	`unsortedArr[i] = rand() % 100;`
	
	`}`
	`displayAll(unsortedArr);`

	``
	void displayAll(int arr[]) {
	    for (int i = 0; i < maxSize; i++) {
	        std::cout << arr[i] << "";
	        std::cout << " ";
	
	    }
	    std::cout << std::endl;
	}
	``
	
	Output:
	``
	"C:\Users\Dread\Desktop\cpe 010\7.1\notes.exe"
	78 8 67 56 7 40 52 69 12 38 27 48 75 7 9 81 28 84 99 87 69 92 91 69 6 12 79 44 32 55 31 90 94 86 22 29 41 99 43 55 31 26
	22 44 35 96 84 6 99 16 39 5 76 22 83 43 45 45 44 10 69 0 25 48 54 49 68 57 22 38 7 7 61 76 35 6 79 92 43 99 20 11 42 19
	84 25 85 75 4 81 47 46 98 65 50 33 68 60 39 27
	``

Observations	The output displays an unsorted random number that has a maximum size of 100. Furthermore, I also used the display function that I created to display all the numbers generated in the loop.

Table 7-1. Array of Values for Sort Algorithm Testing

Code + Console Screenshot	Code:
	``
	#ifndef SORTINGALGO_H
	#define SORTINGALGO_H
	
	#include <iostream>
	
	template <typename T>
	void bubbleSort(T arr[], int maxSize) {
	    for (int i = 0; i < maxSize; i++) {
	        for (int j = i + 1; j < maxSize; j++) {
	            if (arr[j] < arr[i]) {
	                std::swap([&] arr[i], [&] arr[j]);
	            }
	        }
	    }
	}
	
	``
	
	Output:

	Bubble sort 0 4 5 6 6 6 7 7 7 7 8 9 10 11 12 12 16 19 20 22 22 22 22 25 25 26 27 27 28 29 31 31 32 33 35 35 38 38 39 39 40 41 42 43 43 43 44 44 44 45 45 46 47 48 48 49 50 52 54 55 55 56 57 60 61 65 67 68 68 69 69 69 69 75 75 76 76 78 79 79 81 81 83 84 84 84 85 86 87 90 91 92 92 94 96 98 99 99 99 99
Observations	The output displayed the sorted numbers because of bubble sort. Moreover, the outer loop of bubble sort tracks each position in the array,and while the inner loop compares and swaps the elements to be in order. This has n passes amount of time, and has a worst and best case time complexity of O(n^2)

Table 7-2. Bubble Sort Technique

Code + Console Screenshot	Code:
	21 template <typename T>
	22 int routine_smallest(T arr[], int k, int arrSize) {
	23 int position = k;
	24
	25 T smallestElement = arr[k];
	26
	27 for (int j = k+1; j < arrSize; j++) {
	28 if (arr[j] < smallestElement) {
	29 smallestElement = arr[j];
	30 position = j;
	31 }
	32 }
	33 return position;
	34 }
	35

	``cpp
	template <typename T>
	void selectionSort(T arr[], int arrSize) {
	    int min;
	
	    for (int i = 0; i < arrSize; i++) {
	        min = routine_smallest(arr, i, arrSize);
	        std::swap(arr[i], arr[min]);
	
	    }
	}
	``
	Output:
	``
	Selection Sort
	0 4 5 6 6 6 7 7 7 7 8 9 10 11 12 12 16 19 20 22 22 22 22 25 25 26 27 27 28 29 31 31 32 33 35 35 38 38 39 39 40 41 42 43
	43 43 44 44 44 45 45 46 47 48 48 49 50 52 54 55 55 56 57 60 61 65 67 68 68 69 69 69 75 75 76 76 78 79 79 81 81 83 84
	84 84 85 86 87 90 91 92 92 94 96 98 99 99 99 99
	``

Observations	The output displays sorted numbers. Furthermore, routine_smallest goes through all the unsorted arrays to find the minimum value to swap. While the loop in the selectionSort finalized each sorted position. In connection to this the selection sort has n passes amount of times and its best and worst case time complexity is O(n^2).

Table 7-3. Selection Sort Algorithm

Code + Console Screenshot	Code:
	``cpp
	template <typename T>
	void insertionSort(T arr[], int arrSize) {
	    int k = 1, j;
	    T temp;
	
	    while (k < arrSize) {
	        temp = arr[k];
	        j = k - 1;
	        while (j >= 0 && temp <= arr[j]) {
	            arr[j + 1] = arr[j];
	            j--; // back tracking part
	        }
	        arr[j + 1] = temp;
	        k++;
	    }
	}
	``

	Output: Insertion Sort 0 4 5 6 6 6 7 7 7 7 8 9 10 11 12 12 16 19 20 22 22 22 22 25 25 26 27 27 28 29 31 31 32 33 35 35 38 38 39 39 40 41 42 43 43 43 44 44 44 45 45 46 47 48 48 49 50 52 54 55 55 56 57 60 61 65 67 68 68 69 69 69 69 75 75 76 76 78 79 79 81 81 83 84 84 84 85 86 87 90 91 92 92 94 96 98 99 99 99 99
Observations	The output displays other sorted numbers because of the Insertion sort algorithm. Moreover, the first loop iterates through the unsorted items into temp, while the second while loop moves all elements that are greater than or equal to temp one position to the right until temp is inserted in its correct index. Furthermore, insertion sort has n -1 passes at a time, and has a worst case time complexity of O(n^2) and a best case of O(n).

Table 7-4. Insertion Sort Algorithm

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Pseudocode of Algorithm:

START

INITIALIZE string array candidates[5] with candidate names

DECLARE array votes of size 100

SEED random number generator

FOR i = 0; i < maxVotes; i++ DO

  votes[i] <- (RANDOM_INTEGER() MOD 5) + 1

END FOR

CALL insertionSort(votes, maxVotes)

PRINT "Votes (Sorted):"

FOR int i = 0; i < maxVotes; i++ DO

```
    PRINT votes[i]
        IF (i + 1) MOD 20 == 0 THEN
            PRINT NEWLINE
        END IF
END FOR
INITIALIZE array countVotes[numCand] = 0
FOR int i = 0; i < maxVotes; i++ DO
  candidateID <- votes[i]
  countVotes[candidateID] <- countVotes[candidateID] + 1
END FOR
SET highestVotes = -1
   SET winnerIndex = -1
PRINT "Candidate ID  Candidate Name Vote Count"
FOR int i = 0; i < numCand; i++ DO
  PRINT "Candidate " + i, candidates[i], countVotes[i]
    IF countVotes[i] > highestVotes THEN
        highestVotes <- countVotes[i]
        winnerIndex <- i
    END IF
END FOR
DISPLAY “Winner: “
END
```

Code:

```cpp
#include <iostream>
#include <cstdlib>
#include <time.h>
#include "sortingAlgo.h"
#include <string>
#include <iomanip>

const int maxVotes = 100;
const int numCand = 5;

std::string candidates[numCand] = {

    "Bo Dalton Capistrano",
    "Cornelius Raymon Agustin",
    "Deja Jayla Banaga",
    "Lalla Brielle Yabut",
    "Franklin Relano Castro"
};

int main() {
    srand((unsigned int)time(NULL));

    int votes[maxVotes];

    for (int i = 0; i < maxVotes; i++) {
        votes[i] = (rand() % numCand) + 1;
    }
    //sort
    insertionSort(votes, arrSize: maxVotes);

    std::cout <<" Votes (Sorted) " << std::endl;

    for (int i = 0; i < maxVotes; i++) {
        std::cout << votes[i] << " ";
        if ((i+1) % 20 == 0) { // just for organizing
            std::cout << std::endl;
        }
    }
    std::cout << "\n";
```

```
    std::cout << "\n";

    int countVotes[numCand] = {0};
    for (int i = 0; i < maxVotes; i++) {
        countVotes[votes[i] - 1]++;
    }

    std::cout << std::left << std::setw(n:15) << "Candidate #";
    std::cout << std::setw(n:30) << "Candidate Name";
    std::cout << std::setw(n:15) << "Vote Count" << "\n";

    int highestVotes = -1;
    int winnerIndex = -1;

    for (int i = 0; i < numCand; i++) {
        std::cout << std::left << std::setw(n:15) << ("Candidate " + std::to_string(val:i + 1));
        std::cout << std::setw(n:30) << candidates[i];
        std::cout << std::setw(n:15) << countVotes[i] << "\n";

        if (countVotes[i] > highestVotes) {
            highestVotes = countVotes[i];
            winnerIndex = i;
        }
    }
    std::cout << " "<<std::endl;

    std::cout << "WINNER: Candidate " << (winnerIndex + 1) << " - ";
    std::cout << candidates[winnerIndex] << " with ";
    std::cout << highestVotes << " votes!\n";

    return 0;

}
```

Output Console Showing Sorted Array	Manual Count	Count Result of Algorithm
Votes (Sorted) 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5	1 = 23 2 = 19 3 = 20 4 = 17 5 = 21 Winner: candidate #1	Candidate # Candidate Name Vote Count Candidate 1 Bo Dalton Capistrano 23 Candidate 2 Cornelius Raymon Agustin 19 Candidate 3 Deja Jayla Banaga 20 Candidate 4 Lalla Brielle Yabut 17 Candidate 5 Franklin Relano Castro 21 WINNER: Candidate 1 - Bo Dalton Capistrano with 23 votes! Process finished with exit code 0

Votes (Sorted) 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5	1 = 21 2 = 19 3 = 15 4 = 26 5 = 19 Winner : Candidate 4	Candidate # Candidate Name Vote Count Candidate 1 Bo Dalton Capistrano 21 Candidate 2 Cornelius Raymon Agustin 19 Candidate 3 Deja Jayla Banaga 15 Candidate 4 Lalla Brielle Yabut 26 Candidate 5 Franklin Relano Castro 19 WINNER: Candidate 4 - Lalla Brielle Yabut with 26 votes!
Votes (Sorted) 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5	1 = 20 2 = 19 3 =16 4 = 22 5 = 23 Winner : Candidate 5	Candidate # Candidate Name Vote Count Candidate 1 Bo Dalton Capistrano 20 Candidate 2 Cornelius Raymon Agustin 19 Candidate 3 Deja Jayla Banaga 16 Candidate 4 Lalla Brielle Yabut 22 Candidate 5 Franklin Relano Castro 23 WINNER: Candidate 5 - Franklin Relano Castro with 23 votes!

Justification:

- I chose insertion sort because it is the most efficient sorting algorithm to use for this activity. It is because it has the best case time complexity of O(n), if the votes in the array are already mostly sorted.

Question: Was your developed vote counting algorithm effective? Why or why not?

- After completing the supplementary activity, I can say that the vote counting algorithm is effective because it successfully tallied all votes and clustered the numbers for easy verification for manual calculations. Furthermore the candidate number is also the same as the numbers from 1-5 for easy assigning.

Conclusion:

- For this activity, I learned about different types of sorting algorithms, and created C++ codes for the implementation of selection, bubble, and insertion sort. Moreover, the worst case time complexity of all the sorting algorithms that I created is O(n^2) because of having a nested loop. In addition to this, I also used my header file for the sorting algorithm in my supplementary activity. Lastly, I strive to learn more and to be a better c++ programmer.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Gemini**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Explanation of concepts, code assistance**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **I used gemini so that I can understand more about each of the sorting algorithms and to guide me during coding.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.